



DYNAMODB

Il key-value che non ti aspetti



INDICE

- **DYNAMODB in THRON**
- **ARCHITETTURA DI BASE**
- **SCHEMA**
- **INDICI**
- **LOCK**
- **TRANSAZIONI**
- **CHANGE STREAM + OUTBOX**
- **SINGLE TABLE**
- **CONSIDERAZIONI**



RELATORI



Luca Agostini
Software Architect
@ THRON

THRON

THRON è una soluzione **SaaS** per gestire contenuti multimediali (DAM) e relativi dati di prodotto (PIM)

I contenuti sono utilizzati

- all'interno dell'interfaccia amministrativa di THRON
- su portali verticali (es: sito web privato)
- sui canali di comunicazione del cliente (es: sito web pubblico)

THRON deve quindi essere in grado

- **scalare** al crescere dell'uso che fa un singolo cliente potendo gestire
 - un numero crescente di entità
 - un carico crescente
- scala al crescere dei **clienti**
- mantenere sempre **performance** elevate

DYNAMODB in THRON

DynamoDB usato storicamente principalmente per

- gestire configurazioni
- code di job da eseguire
- entità semplici

Abbiamo apprezzato

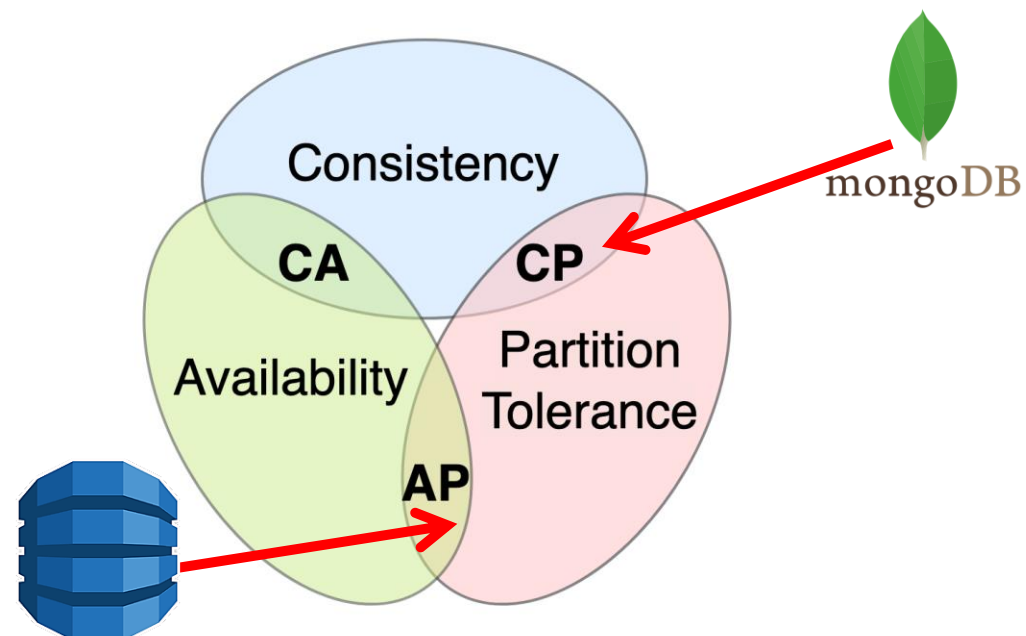
- database **SaaS**
 - nessuna infrastruttura da gestire
- integrazioni con **CI/CD**
 - ben integrato con CDK
- **pay-per-value**
 - con il billing on-demand paghi quello che usi
- affidabilità
- tempo di accesso **single digit**

Abbiamo quindi deciso di metterlo alla prova per dei nuovi sviluppi che richiedevano qualcosa di più complesso

CAP Theorem

In un sistema distribuito non è possibile garantire contemporaneamente tutte e tre le seguenti caratteristiche:

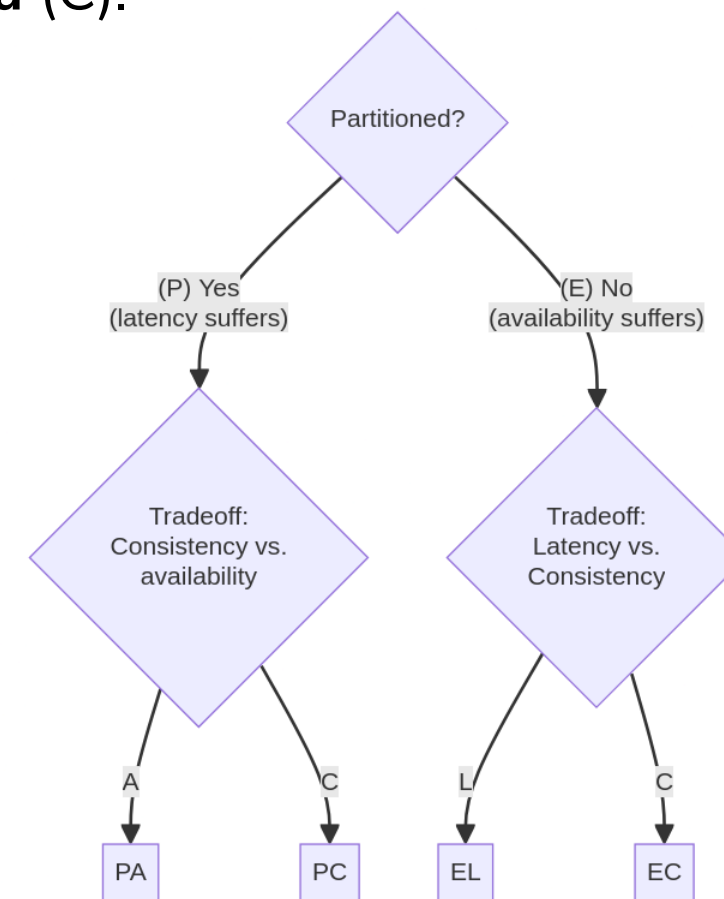
- consistenza (C)
- disponibilità (A)
- tolleranza al partizionamento (P)



PACELC Theorem

In un sistema distribuito, in caso di partizionamento (P), si deve scegliere tra **disponibilità (A)** e **consistenza (C)** (CAP theorem).

Altrimenti (E), quando il sistema lavora senza partizionamento si deve scegliere tra **latenza (L)** e **consistenza (C)**.



ARCHITETTURA DI BASE

Si basa su due principi di scaling

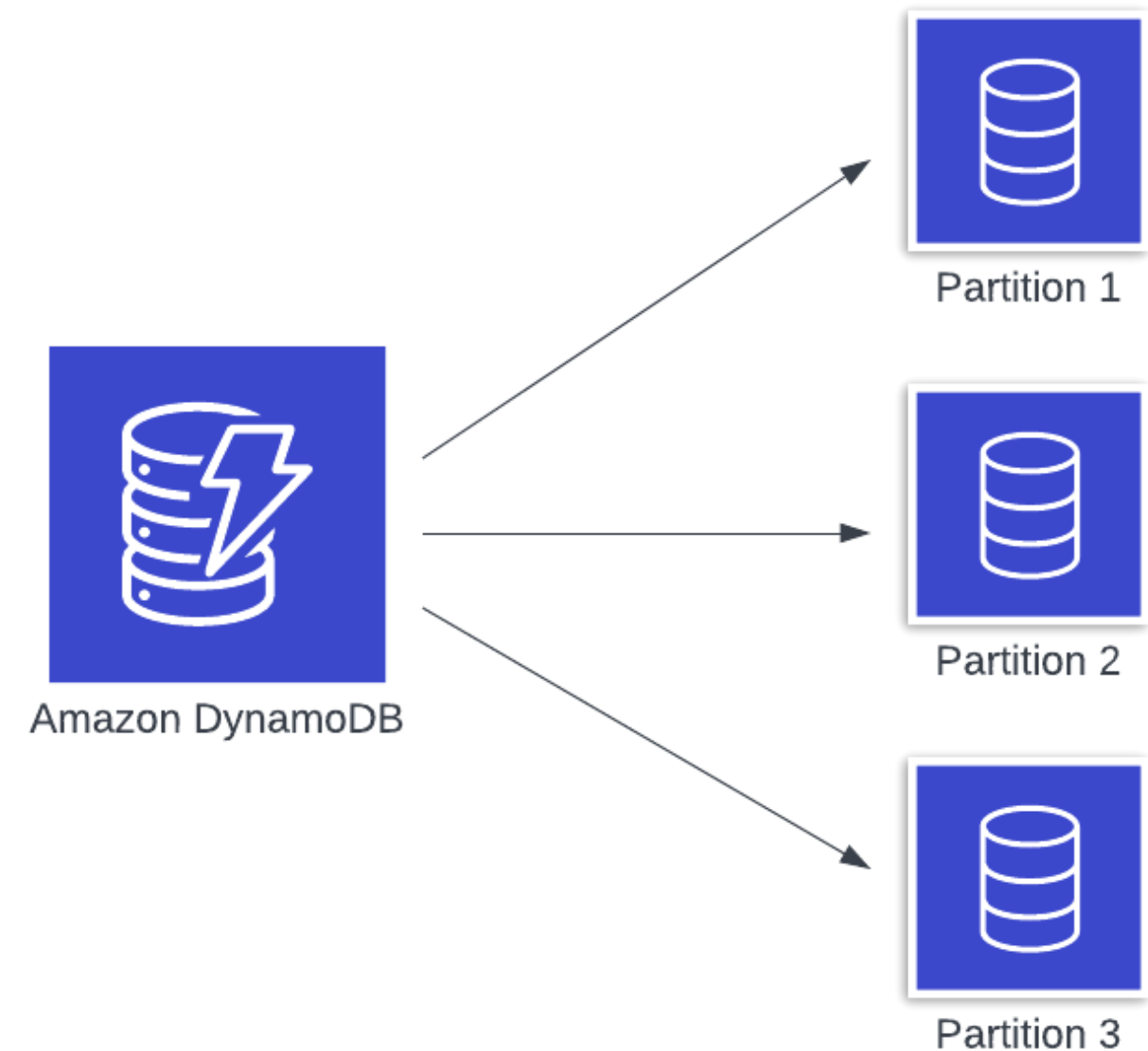
- **Partizionamento** per lo scaling orizzontale
- **B-Tree** per una ricerca e inserimento efficiente

<https://aws.amazon.com/it/blogs/database/single-table-vs-multi-table-design-in-amazon-dynamodb/>

DynamoDB wants to **expose reality** to you, so that you can make the right decision for your application's needs.

Most databases provide an abstraction over the low-level bits. These abstractions make it **easier** for you to query your data in flexible ways, but **they also hide important details from you.**

Because these details are hidden, **these databases can scale in unpredictable** ways or make it difficult to understand what your database will cost as usage grows.



SCHEMA

Lo schema deve essere progettato **avendo chiari quali saranno i pattern di accesso** e considerando i **limiti** di DynamoDB

Ogni item (max 400KB)

- Partition Key (**PK**, al massimo **2048 byte**)
 - ogni partizione in DynamoDB è progettata per erogare di **default** al massimo di 3,000 RRU/s (1 RRU = 4KB consistent read) e 1,000 WRU/s (1 WRU = 1KB)
 - deve essere progettata al fine di non creare partizioni «hot» che causa throttling e rallentamento del sistema
 - nelle query **va sempre indicato in «exact match»**
 - **ogni partizione può contenere un numero potenzialmente illimitato di item**
- Sort Key (**SK** opzionale, al massimo **1024 byte**)
 - si possono avere condizioni tipo «**inizia per**» o «**maggiore/minore di**» o «**between**»
- la **chiave primaria** è la coppia PK e SK (non modificabile)

SCHEMA 1

Esempio di uno schema di una gestione utenti

Primary key		Attributes		
Partition key: TenantId	Sort key: Username			
tenant1	giuseppe.mazzini	FirstName	LastName	Mobile
		Giuseppe	Mazzini	555-0102
	mario.rossi	FirstName	LastName	Mobile
		Mario	Rossi	555-0101
tenant2	attilio.bandiera	FirstName	LastName	Mobile
		Attilio	Bandiera	555-0202
	emilio.bandiera	FirstName	LastName	Mobile
		Emilio	Bandiera	555-0203
	mario.rossi	FirstName	LastName	Mobile
		Mario	Rossi	555-0201

SCHEMA 2

Esempio di uno schema per la gestione utenti, con PK e SK invertiti

Primary key		Attributes	
Partition key: Username	Sort key: T...		
mario.rossi	ena 1	First Name	LastName
	Sort key: T...	Phone	555-0101
giuseppe.mazzini	ant2	First Name	LastName
	Sort key: T...	Phone	555-0201
attilio.bandiera	ena 1	First Name	LastName
	Sort key: T...	Phone	555-0102
emilio.bandiera	Sort key: T...	First Name	LastName
	Sort key: T...	Phone	555-0202
	Sort key: T...	First Name	LastName
	Sort key: T...	Phone	555-0203

SCHEMA 3

Altro esempio di uno schema per la gestione utenti

Primary key Partition key: PK	Attributes				
tenant1#mario.rossi	TenantId	Username	FirstName	LastName	Phone
	tenant1	mario.rossi	Mario	Rossi	555-0101
tenant1#giuseppe.mazzini	TenantId	Username	FirstName	LastName	Phone
	tenant1	giuseppe.mazzini	Giuseppe	Mazzini	555-0102
tenant2#mario.rossi	TenantId	Username	FirstName	LastName	Phone
	tenant2	mario.rossi	Mario	Rossi	555-0201
tenant2#attilio.bandiera	TenantId	Username	FirstName	LastName	Phone
	tenant2	attilio.bandiera	Attilio	Bandiera	555-0202
tenant2#emilio.bandiera	TenantId	Username	FirstName	LastName	Phone
	tenant2	emilio.bandiera	Emilio	Bandiera	555-0203

INDICI LSI

Sono degli indici che permettono di interrogare i dati di una partizione ordinati in modo diverso

- la **partition key** è la stessa della tabella
- la **sort key** è diversa
- letture sia **strong** (costo doppio 🏠 🏠) che **eventually consistent**
- per ogni LSI, si possono memorizzare **fino a 10GB** di dati per ogni partizione
- ci possono essere al massimo 5 LSI per tabella e vanno specificati **in fase di creazione** della tabella

<https://aws.amazon.com/it/about-aws/whats-new/2013/04/18/amazon-dynamodb-announces-local-secondary-indexes/>

Today, local secondary indexes must be defined at the time you create your DynamoDB tables. *In the future, we plan to provide you with an ability to add or drop LSI for existing tables.*



INDICI GSI

Sono degli indici **sparsi** che permettono di interrogare i dati senza dover usare la «chiave primaria»

- richiedono di specificare una diversa **partition key** e **sort key** (non richiedono univocità)
- solitamente PK o SK sono campi composti artificialmente con la **concatenazione di valori**
- solo letture **eventually consistent** 👻 (solitamente <100ms)
- ci possono essere al **massimo 20 GSI** per tabella. È un limite **soft** (prima di alzarlo fatti due domande, CQRS? 🤔)
- le scritture dei dati sugli indici consumano write capacity (attenzione ai costi 💰 💰 💰)
- si possono proiettare nell'indice solo i dati che servono 🙌
- si possono creare e cancellare solo **uno alla volta** 🙌
- la creazione (ovviamente!) **NON E' Istantanea**: se rilasci contemporaneamente tramite CI/CD sia la creazione che l'uso dell'indice, le query falliranno fino alla completa creazione dell'indice 💩

INDICI

Indice secondario (da schema 3):

Primary key		Attributes			
Partition key: TenantId	Sort key: Username				
tenant1	giuseppe.mazzini	PK	FirstName	LastName	Phone
		tenant1#giuseppe.mazzini	Giuseppe	Mazzini	555-0102
	mario.rossi	PK	FirstName	LastName	Phone
		tenant1#mario.rossi	Mario	Rossi	555-0101
tenant2	attilio.bandiera	PK	FirstName	LastName	Phone
		tenant2#attilio.bandiera	Attilio	Bandiera	555-0202
	emilio.bandiera	PK	FirstName	LastName	Phone
		tenant2#emilio.bandiera	Emilio	Bandiera	555-0203
	mario.rossi	PK	FirstName	LastName	Phone
		tenant2#mario.rossi	Mario	Rossi	555-0201

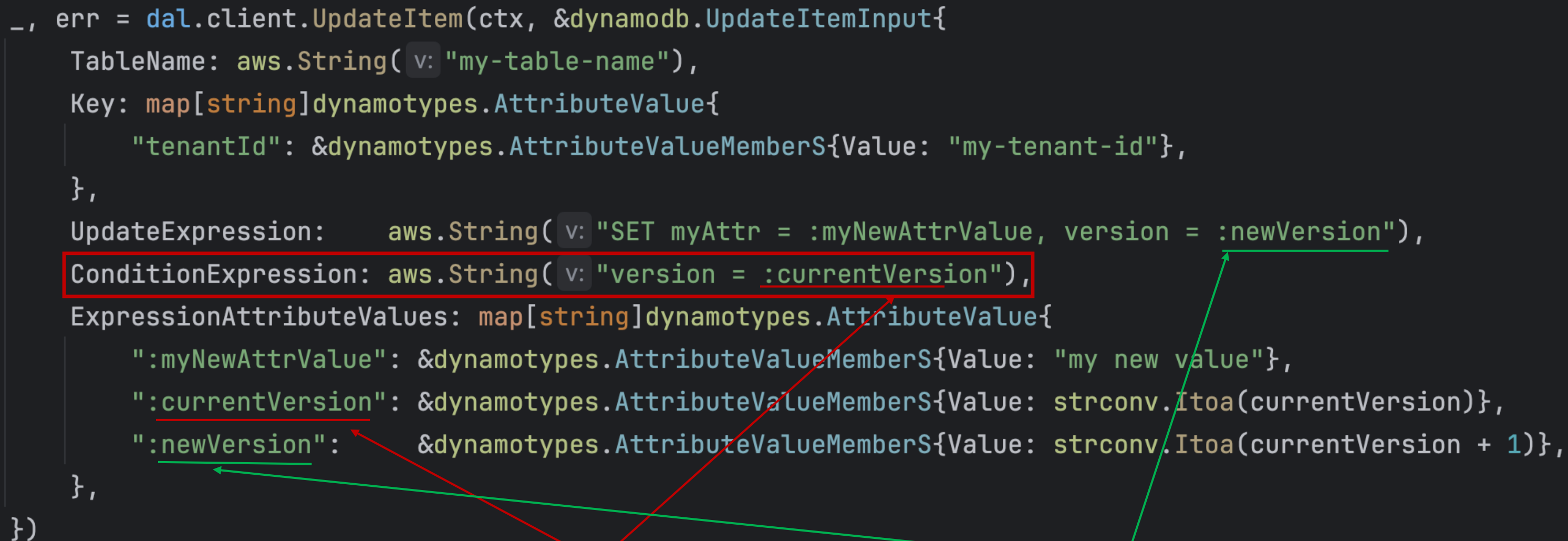
INDICI

- gli indici sono limitati a **due attributi**: un PK ed un SK e vanno gestiti «a mano». Se si prende un approccio **ESR** (Equality Sort Range) negli indici la PK è la E, SK è Sort ed il range è su attributi a piacere
- non puoi creare **indici su array**
- **la dimensione degli attributi PK e SK** sono limitati anche se sufficienti per la maggior parte dei casi d'uso
- **query language limitato**, con PartiQL c'è stato un avvicinamento della sintassi al SQL, ma la potenza espressiva è la medesima
- non c'è un **query planner**

OPTIMISTIC LOCK

Non ci sono meccanismi nativi di lock. Si favoriscono meccanismi di optimistic lock: le scritture possono prevedere delle espressioni condizionali in scrittura

```
_, err = dal.client.UpdateItem(ctx, &dynamodb.UpdateItemInput{
    TableName: aws.String(v: "my-table-name"),
    Key: map[string]dynamodb.AttributeValue{
        "tenantId": &dynamodb.AttributeValueMemberS{Value: "my-tenant-id"},
    },
    UpdateExpression: aws.String(v: "SET myAttr = :myNewAttrValue, version = :newVersion"),
    ConditionExpression: aws.String(v: "version = :currentVersion"),
    ExpressionAttributeValues: map[string]dynamodb.AttributeValue{
        ":myNewAttrValue": &dynamodb.AttributeValueMemberS{Value: "my new value"},
        ":currentVersion": &dynamodb.AttributeValueMemberS{Value: strconv.Itoa(currentVersion)},
        ":newVersion": &dynamodb.AttributeValueMemberS{Value: strconv.Itoa(currentVersion + 1)},
    },
})
```



Vecchio valore

Nuovo valore

TRANSAZIONI

Le scritture possono essere effettuate in modo atomico

- fino a 100 item differenti (anche in tabelle diverse)
- fino a 4MB di payload massimo
- se ci sono più transazioni che stanno scrivendo negli stessi dati, si riceve un errore **TransactionCanceledException**
- Le azioni supportate sono
 - **Put**: crea o aggiorna un item in modo condizionale o meno
 - **Update**: aggiorna o crea un item in modo condizionale o meno
 - **Delete**: elimina un item
 - **ConditionCheck**: verifica che un item esista o che abbia una specifica proprietà

La propagazione della transazione su GSI o stream può avvenire in mezzo ad altre operazioni

Quando si effettua una transazione, il costo delle chiamate raddoppia   .

CHANGE STREAM + OUTBOX

Change stream: sequenza dei cambiamenti che avvengono in una tabella

Outbox: è un pattern che permette di **emettere eventi e scrivere un nuovo stato a DB in modo transazionale.**

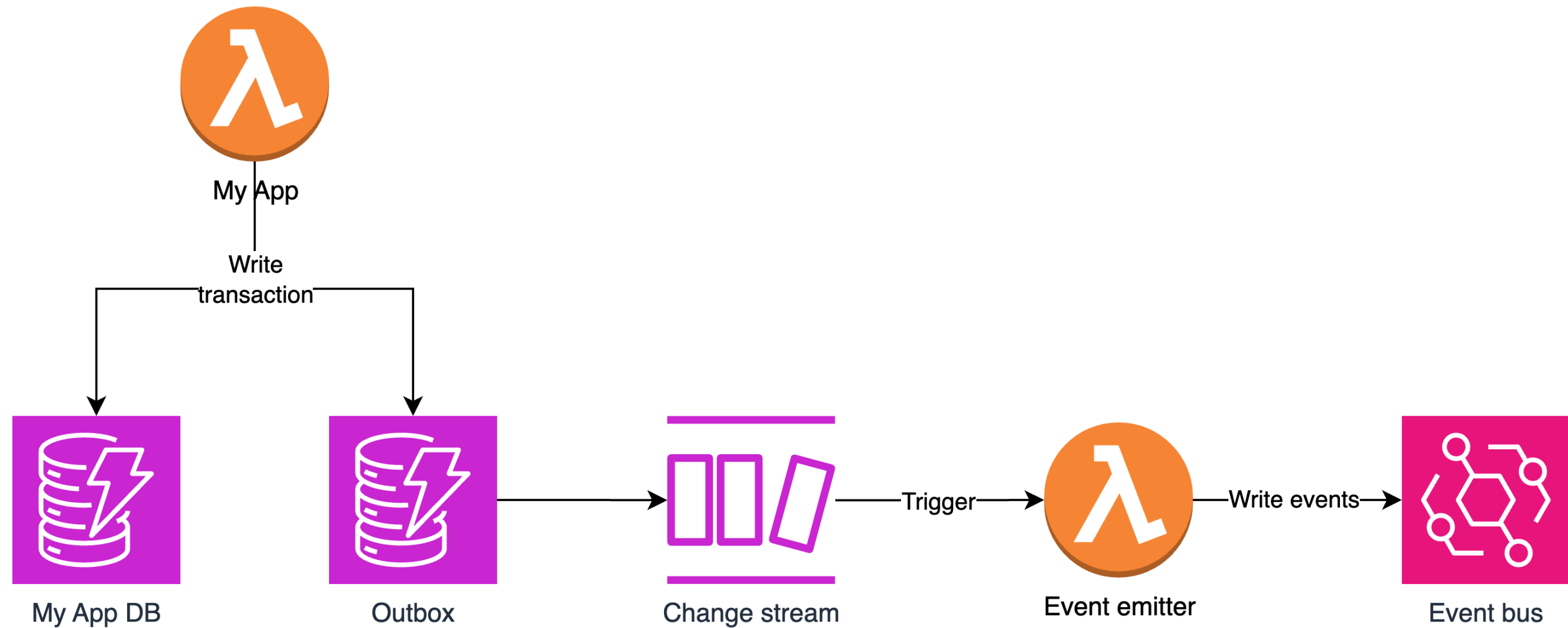
Il non emettere un evento in modo transazionale alla scrittura porta a due scenari

- scrivo i dati a database e POI invio l'evento 
 - se ci sono **problemi nell'invio dell'evento**, perdo l'evento e quindi i sistemi che lo consumano sono disallineati
- invio l'evento e POI scrivo i dati a database 
 - se ci sono **problemi nello scrivere il dato**, l'evento è riferito a fatti mai accaduti

Obiezioni

- ma quante volte vuoi che succeda che non riesca ad inviare l'evento ma ho scritto a database?
- perché non event sourcing?

CHANGE STREAM + OUTBOX



SINGLE TABLE DESIGN

Tecnica per salvare diverse entità su di una stessa tabella superando alcuni limiti di dimensione

Primary key		Attributes			
Partition key: PK	Sort key: S				
tenant1#mario.rossi	email#0	Email			
		mario.rossi@example.com			
	email#1	Email			
		mario.rossi.casa@example.com			
	email#2	Email			
		mario.rossi.work@example.com			
	root	TenantId	Username	FirstName	LastName
		tenant1	mario.rossi	Mario	Rossi

CONSIDERAZIONI

Tutti i modelli sono errati

...qualcuno però è utile (cit.)

...qualcuno è anche pericoloso (cit.)

CONSIDERAZIONI

DynamoDB è uno strumento potente per gestire dati in ambienti con carico variabile grazie all'affidabilità, la scalabilità e la possibilità di scegliere il livello di consistenza

La combinazione della capacità **transazionale**, dell'**optimistic lock** e del **change stream** permette di gestire in modo transazionale l'invio dell'evento e la scrittura del dato in modo semplice

Richiede un design preliminare per capire se lo strumento è adatto al **caso d'uso**

I dati devono essere memorizzati in base ai **pattern di accesso**

Lecture complesse vanno eseguite su altri database (**CQRS**) o valutati altri strumenti

Porre molta attenzione ai diversi limiti del servizio: dimensione item, dimensione transazione, numero item per transazione, numero indici, etc...



WE ARE HIRING



[https://www.thron.com/en/
company/work-with-us/](https://www.thron.com/en/company/work-with-us/)



THRON

Now you know how